

High-Performance Extensible Cellular Automata Engine

Michael Rastetter, 28.05.2026

Konzeption und Implementierung einer modularen,
parallelisierten Simulations-Engine für zelluläre
Automaten als performante Konsolenanwendung.

Technische Dokumentation

Management Summary

Diese technische Dokumentation beschreibt die Konzeptionierung und Implementierung einer hochoptimierten, thread-sicheren Simulations-Engine für *Conway's Game of Life* unter .NET. Das primäre Ziel des Projekts bestand darin, die Grenzen der CPU-Auslastung bei paralleler Datenverarbeitung auszuloten und einen vollständig allokatonsfreien Simulationszyklus zu realisieren.

Durch den Einsatz moderner .NET-Mittel wie permanent im Hintergrund operierenden, dedizierten Worker-Threads, einer blockierungsfreien Barriere-Synchronisation (Barrier-Struktur), feingranularem Thread-Locking („Lock“-Objekte), Cache-line-optimiertem Speicherlayout (Linear-Arrays) sowie dem Double-Checked Locking Pattern beim Statistik-Tracking konnte die Engine erfolgreich gegen Race Conditions und False Sharing abgesichert werden. In praktischen Validierungstests auf einem Standard-Bürolaptop konnte die Effizienz dieser Architektur eindrucksvoll nachgewiesen werden, indem Spitzenwerte von deutlich über einer Milliarde Zellanalysen pro Sekunde erreicht wurden.

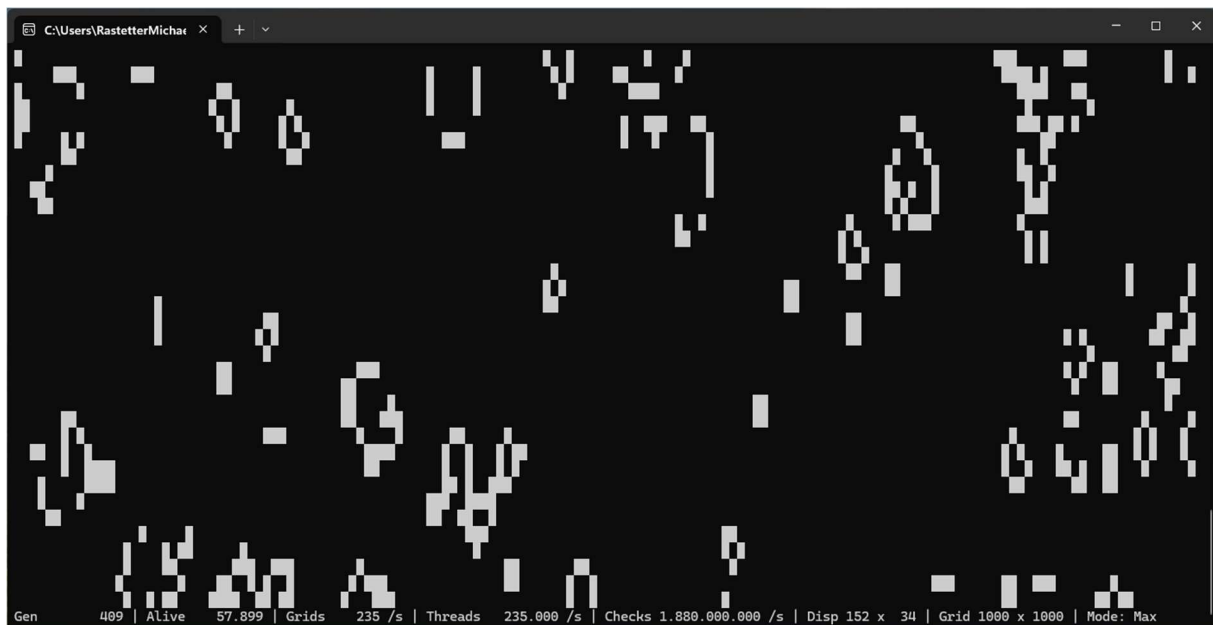


Abbildung: 1,88 Milliarden Zellanalysen pro Sekunde auf einer Intel i5 Mobile CPU

Inhalt

Management Summary.....	2
1 Einleitung und Zielsetzung.....	5
1.1 Projektbeschreibung.....	5
1.2 Motivation und Zielsetzung.....	5
1.3 Zielgruppe und Vorkenntnisse	6
1.4 Formale Beschreibung	6
Zellulärer Automat.....	6
Topologische Randbedingungen	7
Regelwerke	7
2 Anforderungsanalyse (Requirements).....	8
2.1 Funktionale Anforderungen	8
2.2 Nicht-funktionale Anforderungen.....	9
3 Systemarchitektur und Design	11
3.1 High-Level Architektur	11
3.2 Klassendesign (Domänenmodell)	11
Ablaufsteuerung und Zustandsmaschine	12
Der Daten- und Rechenkern	12
Das Strategie-Muster für Evolutionsbiologie	13
3.3 Datenfluss und Thread-Interaktion.....	13
4 Technische Implementierung und Performance-Optimierung	15
4.1 Dediziertes Multithreading-Modell (0-Alloc Thread-Pooling).....	15
4.2 Das Allokationsfreie Speicher-Design	15
Back-Buffer-Verfahren und Pointer-Swapping	15
Vermeidung des Heaps durch Generics und Struct-Constraints.....	16
Lambda-Hygiene und Vermeidung verdeckter Heap-Instanziierungen	16
4.3 Branchless Bit-Manipulation im Detail	17
4.4 Thread-Synchronisation.....	17
Die Lock-Hierarchie	18
Double-Checked Locking.....	18
Barriere-Synchronisation	18

Lokale Akkumulation	19
5 Qualitätssicherung und Testing	20
5.1 Teststrategie und Testbarkeit	20
5.2 Unit-Testing der Kernkomponenten	20
Verifikation der Regelwerke und Bit-Masken	20
Validierung der topologischen Randbedingungen	20
Robustheit des RLE-Parsers	21
5.3 Multithreading- und Concurrency-Tests	21
5.4 Memory-Profiling und Verifikation der Allokationsfreiheit	22
6 Performance-Analyse und Benchmarks	24
6.1 Testumgebung und Methodik	24
6.2 Durchsatzanalyse und Skalierungsverhalten	24
6.3 Weitergehende Tests und Erwartungen	25
Effizienz der Branchless-Bitmanipulation	25
Auswirkung des JIT-Inlinings durch Struct-Constraints	26
Einfluss der Cache-Lokalität (Grid-Größen-Effekt)	26
7 Fazit und Ausblick	27
7.1 Zusammenfassung	27
7.2 Lessons Learned	27
7.3 Zukünftige Erweiterungen	28
Anhang A: Bedienungsanleitung (User Guide)	29
A.1 Systemanforderungen	29
A.2 Installation und Programmstart	29
A.3 Konfiguration via JSON-Datei	29
A.4 Tastatursteuerung im laufenden Betrieb	31
A.5 System-Statusanzeige (Heads-Up-Display)	32
Anhang B: UML-Diagramm	33

1 Einleitung und Zielsetzung

1.1 Projektbeschreibung

Das vorliegende Projekt umfasst die Konzeptionierung und Realisierung einer softwarebasierten Simulations-Engine für zelluläre Automaten am Beispiel von *Conway's Game of Life* und dessen Modifikation *HighLife*. Die Anwendung ist als plattformunabhängige Konsolenanwendung unter der modernen Laufzeitumgebung .NET 9.0 in C# umgesetzt.

Die primäre Aufgabe der Software besteht darin, zweidimensionale zelluläre Gitter (Grids) nach mathematisch definierten Evolutionsregeln in diskreten Zeitschritten (Generationen) zu berechnen und visuell darzustellen. Die Kernarchitektur trennt hierbei strikt zwischen der mathematischen Berechnungseinheit (**SimulationEngine**), der Benutzerinteraktion und Ablaufsteuerung (**GameController**) sowie der grafischen Ausgabekomponente (**ConsoleRenderer**). Das System unterstützt neben der klassischen, zufallsbasierten Initialisierung auch das Parsen und Laden standardisierter Run-Length-Encoded-Dateien (**.rle**), wodurch komplexe, historisch erforschte Zellmuster exakt reproduziert werden können.

1.2 Motivation und Zielsetzung

Die mathematische Simulation von zellulären Automaten neigt bei steigenden Gitterdimensionen zu einem exponentiell anwachsenden Rechenaufwand. Die Motivation hinter diesem Projekt liegt nicht in der bloßen Abbildung der Spielregeln, sondern im systematischen Ausloten moderner Hardware-Ressourcen und Compiler-Optimierungen zur Erreichung maximaler Berechnungsgeschwindigkeiten.

Das Projekt verfolgt drei fundamentale technische Kernziele:

- **Maximierung des Durchsatzes (High-Performance Parallelism):** Durch den Einsatz hardwarenaher Algorithmen und die geschickte Verteilung der Rechenlast auf alle verfügbaren logischen CPU-Kerne soll eine hocheffiziente Skalierung erzielt werden.
- **Garantie der Allokationsfreiheit (Zero-Alloc):** Zur Laufzeit der Simulationsschleife dürfen keinerlei Garbage-Collection-Zyklen (GC) ausgelöst werden. Das bedeutet, dass der Heap-Speicher während der Berechnungsphase unangetastet bleibt (0 Byte Allokation), um unvorhersehbare Performance-Einbrüche (Stuttering) vollständig zu eliminieren.
- **Thread-Sicherheit und Latenzfreiheit:** Die Entkopplung von Rechen- und Rendering-Zyklen erfordert ein feingranulares, blockierungsfreies Synchronisationskonzept. Die Benutzeroberfläche muss asynchron und

verzögerungsfrei auf Zustandsänderungen reagieren können, ohne Race Conditions zu riskieren.

1.3 Zielgruppe und Vorkenntnisse

Diese Dokumentation richtet sich an Software-Architekten, Systementwickler und Informatiker, die sich mit den Themengebieten Low-Level-Performance-Optimierung, Multithreading und hardwarenaher Softwareentwicklung unter .NET beschäftigen.

Für das Verständnis der nachfolgenden Kapitel werden fundierte Kenntnisse in der objektorientierten und funktionalen Programmierung mit C# vorausgesetzt. Zudem sind grundlegende Kenntnisse über die Funktionsweise des JIT-Compilers (Just-in-Time), CPU-Cache-Architekturen (L1/L2/L3-Caches), Speicherbarrieren (**Volatile**) sowie atomare CPU-Instruktionen (**Interlocked**) empfehlenswert.

1.4 Formale Beschreibung

Zellulärer Automat

Ein zellulärer Automat ist ein diskretes mathematisches Modell zur Simulation räumlich-zeitlicher dynamischer Systeme. Er besteht aus einem regulären, unendlichen oder begrenzten Gitter aus homogenen Zellen. Jede Zelle nimmt zu einem diskreten Zeitpunkt t einen Zustand aus einer endlichen Zustandsmenge an. Die synchrone Aktualisierung aller Zellen erfolgt in diskreten Schritten $t_n \rightarrow t_{n+1}$ streng deterministisch über lokale Nachbarschaftsregeln. Trotz der Einfachheit dieser lokalen Regeln neigen zelluläre Automaten zu komplexem, emergentem Verhalten.

Ein zellulärer Automat wird formal durch ein Quadrupel $A = (G, Z, N, f)$ beschrieben.

1. **Zellularfeld:** Eine räumliche Anordnung $\mathbb{G}$, definiert als ein d -dimensionales Gitter. Im Rahmen dieser Anwendung beschränkt sich das Feld auf ein zweidimensionales, rechtwinkliges Koordinatensystem ($d=2$).
2. **Zustandsmenge** $\mathbb{Z}$: Eine endliche Menge von Zuständen, die eine Zelle annehmen kann. In einer binären Engine gilt $Z = \{0, 1\}$ wobei 0 den Zustand „tot“ (inaktiv) und 1 den Zustand „lebendig“ (aktiv) repräsentiert.
3. **Nachbarschaft** $\mathbb{N}$: Eine geordnete Menge von relativen Koordinaten, die festlegt, welche umliegenden Zellen den Folgezustand einer Zielzelle beeinflussen. Diese Engine implementiert zwei klassische Topologien:
 - a. **Von-Neumann-Nachbarschaft:** Berücksichtigung der 4 orthogonal angrenzenden Nachbarn (Norden, Süden, Westen, Osten).
 - b. **Moore-Nachbarschaft:** Berücksichtigung aller 8 umliegenden Zellen einschließlich der Diagonalen.

4. **Überföhrungsfunktion:** Die lokale Funktion, die den Zustand einer Zelle in der nächsten Generation (t+1) bestimmt.

$$z_i^{t+1} = f(z_i^t, \{z_j^t \mid \forall j \in \mathbb{N}(i)\})$$

Der Folgezustand hängt ausschließlich vom eigenen Zustand und dem kumulierten Zustand der definierten Nachbarschaft N zum Zeitpunkt t ab.

Topologische Randbedingungen

Da ein physischer Speicher im Gegensatz zum mathematischen Modell endlich ist, muss das Verhalten an den Gittergrenzen explizit definiert werden:

- **Endliches Spielfeld** (Bordered / Reflexion): Der Rand wird definitionsgemäß mit permanent inaktiven Zellen (z=0) besetzt. Dies führt bei dynamischen Strukturen an den Grenzen zu Mustereinfrierungen oder destruktiven Reflexionen.
- **Toroidales Spielfeld** (Torus-Verschrankung): Das zweidimensionale Gitter wird topologisch auf die Oberfläche eines Torus (Donut-Form) projiziert. Der rechte Rand grenzt nahtlos an den linken Rand, der untere Rand an den oberen Rand. Zustandsänderungen, die über eine Grenze hinauswandern, treten auf der gegenüberliegenden Seite wieder in das Feld ein.

Regelwerke

Das von John Horton Conway im Jahr 1970 entworfene *Game of Life* ist der bekannteste zweidimensionale zelluläre Automat. Er operiert auf einer Moore-Nachbarschaft (|N| = 8) unter Anwendung dreier Übergangsregeln:

1. **Überleben:** Eine lebende Zelle bleibt lebend, wenn die Summe ihrer lebenden Nachbarn exakt 2 oder 3 beträgt.
2. **Geburt:** Eine tote Zelle wird in der Folgegeneration zum Leben erweckt, wenn sie von exakt 3 lebenden Nachbarn umgeben ist.
3. **Isolierung / Überbevölkerung:** In allen anderen Fällen stirbt die Zelle oder bleibt tot.

Als evolutionäre Variante unterstützt diese Engine das *HighLife*-Regelwerk. Dieses verhält sich identisch zu Conways Regeln, erweitert die Geburtsregel jedoch um den Wert 6. Eine tote Zelle wird hierbei also auch dann lebendig, wenn sie exakt 6 lebende Nachbarn besitzt (B36/S23), was zu stark veränderten, hochdynamischen Replikator-Mustern führt.

Die formale Zustandstransition für das klassische *Game of Life* lässt sich in folgender Wahrheitstabelle zusammenfassen:

Anzahl lebender Nachbarn	0	1	2	3	4	5
Zustand Folgegeneration	Tot	Tot	Gleich	Lebend	Tot	Tot

2 Anforderungsanalyse (Requirements)

2.1 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben das direkt beobachtbare Verhalten der Anwendung. Das System muss folgende Kernfunktionalitäten zur Verfügung stellen:

Zustandssteuerung der Simulation: Der Anwender muss in der Lage sein, die Simulation flexibel über Tastaturbefehle zu steuern. Dies impliziert das nahtlose Umschalten zwischen verschiedenen Ausführungsmodi in Echtzeit:

- *Einzelschrittmodus (Step Mode via F1):* Manuelle Weiterschaltung um exakt eine Generation per Tastendruck.
- *Verzögerte Modi (Slow/Fast via F2/F3):* Gedrosselte Ausführung zur visuellen Verfolgung der Zellpopulationen mit definierten Ruhezeiten (**Thread.Sleep**).
- *Maximalmodus (Max Mode via F4):* Ungebremste Berechnungsausführung unter Volllast aller CPU-Ressourcen.

Dynamische Initialisierung und Muster-Injektion: Die Engine muss zwei unterschiedliche Modi zur Gitter-Generierung unterstützen:

- Generierung eines pseudozufälligen Ausgangszustands basierend auf einer in den Einstellungen definierten Populationsdichte (**Density**).
- Deterministisches Laden und Dekodieren von **RLE**-Dateien unter Einhaltung syntaktischer Spezifikationen (Berücksichtigung von Multiplikatoren, Zeilenumbrüchen „\$“ und dem Dateiendezeichen „!“).

Regelwerks- und Topologiewechsel: Das System muss beim Programmstart konfigurierbar zwischen den Regelwerken *Conway's Game of Life* (B3/S23) und *HighLife* (B36/S23) wechseln können. Zudem muss die Gitter-Topologie sowohl als endliche Welt (mit harten Grenzen) als auch als toroidale Welt (unendliche Umwicklung der Ränder / Donut-Form) berechnet werden können.

Echtzeit-Statistikanzeige: Am unteren Bildschirmrand muss eine permanente, asynchrone angesteuerte Statuszeile ausgegeben werden, welche die aktuellen Metriken (aktuelle Generation, Anzahl lebender Zellen, Berechnungen pro Sekunde, bearbeitete Threads pro Sekunde sowie die aktuellen Gitter- und Fensterdimensionen) fehlerfrei darstellt.

Externe Datei-Konfiguration: Das System muss beim Programmstart in der Lage sein, seine initialen Betriebsparameter dynamisch aus einer externen Konfigurationsdatei im JSON-Format einzulesen. Zu den konfigurierbaren Parametern gehören:

- Die Gitterdimensionen (Breite und Höhe des Spielfelds).

- Das anzuwendende mathematische Regelwerk (Conway oder HighLife) sowie die Nachbarschaftstopologie (Moore oder VonNeumann).
- Die Randbedingungen (endlich/begrenzt oder toroidal).
- Die Ziel-Bildwiederholrate für die grafische Ausgabe.
- Die initiale Populationsdichte bei Zufallgenerierung oder alternativ der Dateipfad zu einer zu ladenden RLE-Musterdatei.
- Das Startverhalten, d.h. der anzuwendende Betriebsmodus und ob ein Hilfebildschirm angezeigt werden soll.

2.2 Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen definieren die qualitativen Kriterien und technischen Leitplanken, unter denen die funktionalen Anforderungen realisiert werden müssen:

Ausführungsgeschwindigkeit und Durchsatz: Die Simulations-Engine muss hochgradig optimiert sein. Auf moderner Consumer-Hardware (z. B. einer Intel i5-CPU) muss die Verarbeitungsgeschwindigkeit bei adäquaten Gittergrößen die Schwelle von **1.000.000.000 (einer Milliarde) Zellanalysen pro Sekunde** überschreiten.

Allokationsprofil (Memory Footprint): Innerhalb der zentralen Simulationsschleife (**SimulationLoop**) sowie im parallelen Rechenkern (**UpdatePatternGeneric**) dürfen während der Ausführung keine Heap-Allokationen stattfinden. Das Erzeugen neuer Objekte, Arrays oder das Entstehen von Closures (durch anonyme Delegaten, also z.B. Lambda-Ausdrücke) ist sowohl auf Zell- als auch auf Schleifensteuerungs-Ebene strikt untersagt. Alle Operationen müssen auf dem Stack oder auf langlebigen, wiederverwendbaren Objekten stattfinden.

Thread-Sicherheit und Synchronisations-Overhead: Die Synchronisation der parallel operierenden Berechnungs-, Rendering- und Eingabe-Threads muss ohne Deadlocks und Race-Conditions erfolgen. Der Synchronisations-Overhead ist dabei so gering wie möglich zu halten; zeitintensive blockierende Locks dürfen die parallele Ausführung der Rechen-Threads zu keinem Zeitpunkt ausbremsen.

Robuste Speicher-Lokalität (Cache-Friendliness): Das zweidimensionale Gitter muss intern zwingend in einem eindimensionalen, flachen Array (**bool[]**) verwaltet werden. Die Schleifenstrukturen müssen so aufgebaut sein, dass sie sequenziell im Speicher fortschreiten, um die Hardware-Pre-Fetcher der CPU optimal zu nutzen und Cache-Misses zu minimieren.

Robustheit und Validierung: Die Anwendung muss Fehleingaben oder korrupte Konfigurationsdateien (z. B. unvollständige JSON-Dateien oder ungültige RLE-

Koordinatenüberschreitungen) isolieren, abfangen und den Anwender mittels qualifizierter Fehlermeldungen informieren, ohne unkontrolliert abzustürzen.

3 Systemarchitektur und Design

3.1 High-Level Architektur

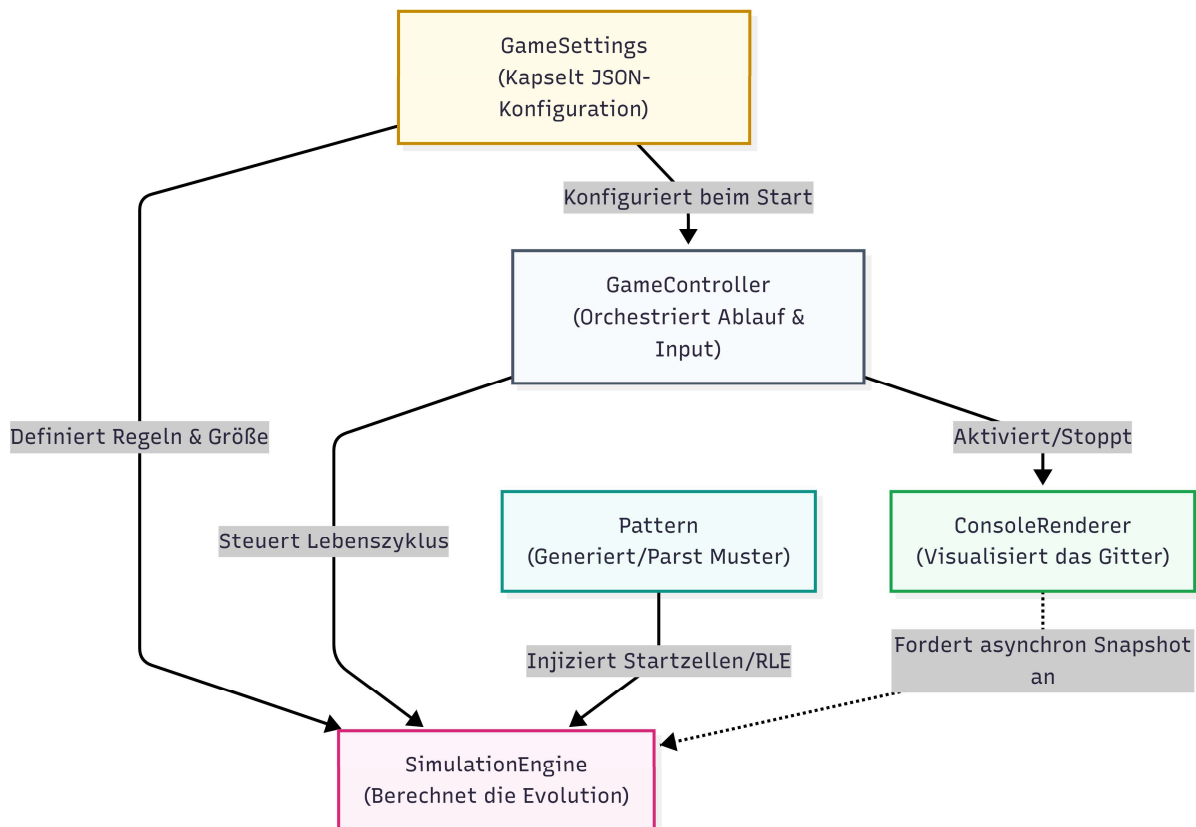
Die Architektur der Gesamtanwendung folgt einem strikt entkoppelten Drei-Säulen-Modell, um maximale Performance bei der Berechnung von der naturgemäß langsamen Konsolenausgabe (I/O-Bound) zu trennen. Das System verteilt seine Aufgaben auf 2+N Threads:

1. **Main-Thread** (Eingabeschleife): Blockiert innerhalb der **InputLoop** des **GameController** durch den synchronen Aufruf von **Console.ReadKey**. Er wartet exklusiv auf Benutzerinteraktionen und steuert den globalen Zustandswechsel.
2. **Render-Thread** (Ausgabe-Engine): Läuft entkoppelt im **ConsoleRenderer** innerhalb der **RenderLoop**. In festen Zeitintervallen ($1000 / \text{_targetFps}$) holt er sich einen konsistenten Schnappschuss der Daten und zeichnet danach (!) das Gitter in die Konsole.
3. **Simulation-Thread** (Rechenkern): Führt die äußere **SimulationLoop** aus. Er besitzt die alleinige Kontrollhoheit und taktet die Berechnung, indem er die im Konstruktor initialisierten **N Hintergrund-Worker-Threads** über eine **Barrier**-Struktur synchronisiert und aufweckt. N ist hierbei die Anzahl der logischen CPU-Kerne.

Durch diese Dreiteilung wird verhindert, dass die langsame Textausgabe der Konsole oder das Warten auf einen Tastendruck den parallelen Rechenkern ausbremsen.

3.2 Klassendesign (Domänenmodell)

Das Klassendesign ist nach den Prinzipien der hohen Kohäsion und losen Kopplung entworfen. Die folgende Übersicht beschreibt die Kernkomponenten des Systems und ihre strategische Aufgabe:



Ablaufsteuerung und Zustandsmaschine

- **Program:** Fungiert als Einstiegspunkt (Bootstrapper). Es lädt die JSON-Konfiguration via **GameSettings**, initialisiert die Kernkomponenten und startet den **GameController**.
- **GameController:** Orchestriert die Lebensdauer der Threads und verwaltet den aktuellen **SimulationMode** (Zustandsmaschine). Er fungiert als Vermittler zwischen Benutzereingabe und der Engine. Er implementiert **IDisposable**, um Thread-Signalisierungsobjekte (**ManualResetEventSlim**) sauber freizugeben.

Der Daten- und Rechenkern

- **SimulationEngine:** Das Herzstück der Anwendung. Sie kapselt die Gitterdaten und steuert die atomaren Schritte der Evolution. Sie verbirgt die komplexen Generics vor der Außenwelt und stellt via Thread-Sicherheits-Barrieren (**Volatile.Read**) konsistente Statistikdaten bereit.
- **GridBuffer:** Eine reine Datenstruktur, die das zweidimensionale Spielfeld in einem flachen, eindimensionalen **bool[]**-Array speichert. Sie bietet einen Indexer für den semantischen Zugriff via **[x, y]**, kapselt jedoch den direkten Array-Zugriff für performante Bulk-Operationen.

- **Pattern:** Eine zustandslose Fabrikklasse (statische Klasse). Sie ist verantwortlich für die Generierung von Zufallsmustern und enthält den komplexen Parser für das Einlesen und Expandieren von Run-Length-Encoded-Dateien (`.rle`).

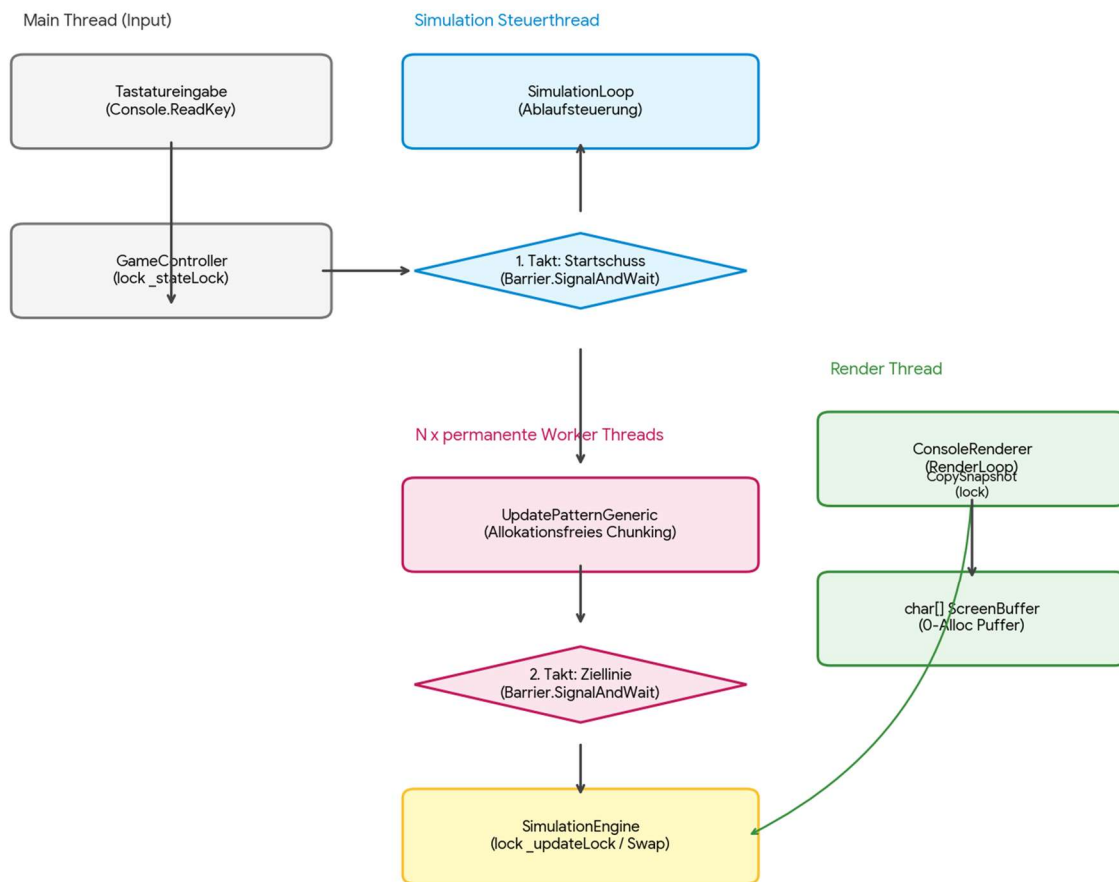
Das Strategie-Muster für Evolutionsbiologie

Um Modifikationen am Regelwerk oder der Nachbarschaft ohne Performance-Verluste zur Laufzeit zu ermöglichen, wird ein striktes Strategie-Muster über Interfaces erzwungen:

- **ICellularRule:** Schnittstelle für das mathematische Regelwerk, implementiert durch `ConwayRule` und `HighLifeRule`.
- **INeighbourhoodStrategy:** Schnittstelle für die Nachbarschaftsanalyse, implementiert durch `MooreNeighbourhood` und `VonNeumannNeighbourhood`.

3.3 Datenfluss und Thread-Interaktion

Der Datenfluss zwischen den Threads erfolgt streng gerichtet und wird durch minimale, spezialisierte Synchronisationspunkte abgesichert:



- Wenn der **Main Thread** eine Taste registriert (z. B. F1 für Schrittmodus), sperrt er kurzzeitig das `_stateLock` im `GameController`, ändert den Modus und signalisiert dem Simulationsthread über `_stepSignal.Set()`, dass ein neuer Rechenschritt erlaubt ist.
- Der **Simulation Thread** liest den Zustand isoliert aus, berechnet die nächste Generation exklusiv im `_updateLock` der `SimulationEngine` und schreibt die Daten in den Back-Buffer (`_nextGrid`). Nach Abschluss erfolgt ein atomarer Zeigertausch (Pointer Swap).
- Der **Render Thread** fordert asynchron eine Kopie des aktuellen Zustands an. Er betritt kurzzeitig das `_updateLock`, um mittels ultraschnellem `Array.Copy` in der Methode `CopySnapshot` die Daten in seinen lokalen `_cellsBuffer` zu spiegeln. Danach verlässt er das Lock sofort wieder, transformiert die Daten im eigenen Thread in einen flachen `StringBuilder` und flusht diesen flackerfrei auf den Bildschirm.

4 Technische Implementierung und Performance-Optimierung

4.1 Dediziertes Multithreading-Modell (0-Alloc Thread-Pooling)

Die Verteilung der Berechnungslast auf die verfügbaren CPU-Kerne erfolgt über ein anwendungsspezifisches Multithreading-Verfahren. Da das standardmäßige `Parallel.For`-Konstrukt von .NET bei hochfrequenten Aufrufen im Mikrosekundenbereich durch die Erzeugung interner Task-Strukturen und Compiler-Closures Heap-Allokationen verursacht, wurde ein vollständig allokatonsfreies Modell implementiert.

Hierbei werden einmalig beim Systemstart exakt so viele Hintergrund-Threads (Thread) initialisiert, wie logische CPU-Kerne (`Environment.ProcessorCount`) vorhanden sind. Diese Threads verbleiben dauerhaft im Speicher und operieren in einer dedizierten Arbeitsschleife (`WorkerLoop`), die über eine systemnahe `Barrier`-Struktur synchronisiert wird

```
private void WorkerLoop(int startY, int endY) {
    try {
        while (!_isDisposed) {
            _barrier.SignalAndWait(); // Warten auf Startschuss
            if (_isDisposed) break;
            _currentStepMethod?.Invoke(startY, endY);
            _barrier.SignalAndWait(); // Rückmeldung an Hauptthread
        }
    } catch (ObjectDisposedException) {}
}
```

Dieses statische Aufteilen (Chunking) stellt sicher, dass jeder CPU-Kern ein exakt zugewiesenes, zusammenhängendes Datensegment im Arbeitsspeicher (Cache-Line-Lokalität) ohne jeglichen Framework-Overhead oder Kontextwechsel-Verluste berechnet.

4.2 Das Allokationsfreie Speicher-Design

Ein zentrales Qualitätsmerkmal der Engine ist die absolute Allokationsfreiheit während des Simulationszyklus. Dadurch wird sichergestellt, dass die automatische Speicherbereinigung (.NET Garbage Collector) niemals anspringen muss, was unvorhersehbare Mikroruckler (GC-Stuttering) verhindert.

Back-Buffer-Verfahren und Pointer-Swapping

Anstatt nach jeder Generation ein neues Array für den Folgezustand zu instanziiieren oder Daten mittels `Array.Copy` zu klonen, nutzt die Engine zwei langlebige Instanzen der Klasse `GridBuffer` (`_currentGrid` und `_nextGrid`), die im Konstruktor

einmalig allokiert werden. Am Ende eines Berechnungszyklus werden lediglich die Objektreferenzen (Zeiger) der beiden Gitter atomar getauscht:

```
(_nextGrid, _currentGrid) = (_currentGrid, _nextGrid);
```

Durch diesen **Pointer-Swap** wird der Schreibpuffer der aktuellen Generation in der nächsten Generation sofort als schreibgeschützter Lesebuffer wiederverwendet, ohne dass auch nur ein einziges Byte im Heap-Speicher bewegt oder instanziiert werden muss.

Vermeidung des Heaps durch Generics und Struct-Constraints

Normalerweise führt die Verwendung von Interfaces in C# zu virtuellen Methodenaufrufen (vtable-Lookups) und beim Zuweisen von Werttypen zu implizitem Boxing auf den Heap. Um dies zu verhindern, deklariert die zentrale Rechenmethode die Regeln und Nachbarschaften als generische Typparameter mit einem expliziten **struct**-Constraint:

```
private void UpdatePatternGeneric<TRule, TStrategy>()  
    where TRule : struct, ICellularRule  
    where TStrategy : struct, INeighbourhoodStrategy
```

Da es sich bei **ConwayRule** und **HighLifeRule** um Strukturen (**struct**) handelt, erzwingt dieser Constraint, dass der JIT-Compiler (Just-In-Time) für jede Kombination (z. B. **<ConwayRule, MooreNeighbourhood>**) einen spezialisierten, hochoptimierten Maschinencode generiert. Der Compiler kennt dadurch die konkrete Implementierung zur Kompilierzeit und kann die Methoden vollständig **inline** in die Schleife einbetten. Es entstehen weder Heap-Allokationen noch die Overheads virtueller Schnittstellenaufrufe.

Lambda-Hygiene und Vermeidung verdeckter Heap-Instanziierungen

Anonyme Funktionen und Lambda-Ausdrücke (=>) bergen in C# ein erhebliches Risiko für verdeckte Heap-Allokationen. Sobald ein Lambda-Ausdruck Variablen aus seinem umschließenden Kontext verwendet (Variable Capturing), generiert der Compiler im Hintergrund eine unsichtbare Zustandsklasse (eine sogenannte DisplayClass). Jedes Mal, wenn der Programmpfad das Lambda erreicht, wird diese Klasse auf dem Heap instanziiert, was zu massiven Allokationsraten im Performance-Pfad führt.

Zur strikten Einhaltung des „Zero-Alloc“-Paradigmas wurde in der Engine eine konsequente Lambda-Hygiene im gesamten Simulationszyklus umgesetzt.

- Die Übergabe der Zeilengrenzen (**startY/endY**) an die Worker-Threads erfolgt beim Anwendungsstart alloktionsfrei über unmanaged **ValueTuple**-Strukturen auf dem Stack.

- Die dynamische Auswahl des Regelwerks wird über einen gecachten, im Konstruktor aufgelösten typsicheren Funktionszeiger (**StepDelegate**) realisiert.

4.3 Branchless Bit-Manipulation im Detail

Innerhalb der Zellberechnung entscheidet sich die Performance der Engine. Standardmäßige **if-else**-Verzweigungen führen auf modernen CPUs bei chaotischen Mustern zu Fehlprognosen der hardwareseitigen Sprungvorhersage (Branch Misprediction), was die CPU-Pipeline leert und Leistungseinbußen zur Folge hat. Die Klassen **ConwayRule** und **HighLifeRule** eliminieren Verzweigungen mathematisch über Bit-Masken und direkte Registermanipulation:

```
public readonly bool CalculateNextState(bool currentState, int
livingNeighbours)
{
    int neighbourBit = 1 << livingNeighbours;
    int stateSign = -Unsafe.As<bool, byte>(ref currentState);
    int mask = BornMask ^ (stateSign & XorMask);
    return (neighbourBit & mask) != 0;
}
```

Die mathematische Funktionsweise im Detail:

1. **neighbourBit = 1 << livingNeighbours**: Erzeugt ein Bit an der Position der gezählten Nachbarn (hat eine Zelle 3 Nachbarn, wird das 3. Bit gesetzt: **00001000**).
2. **Unsafe.As<bool, byte>(ref currentState)**: Liest den Wahrheitswert der Zelle ohne Konvertierungs-Branch direkt als Byte aus dem Register (0 für **false**, 1 für **true**). Durch das unäre Minus (-) entsteht im Zweierkomplement die Bitmaske **00000000** (Zelle tot) oder **11111111** (Zelle lebendig).
3. **BornMask ^ (stateSign & XorMask)**: Schaltet über eine XOR-Verknüpfung dynamisch zwischen der Geburts-Maske (wenn Zelle inaktiv) und der Überlebens-Maske (wenn Zelle aktiv) um.
4. **(neighbourBit & mask) != 0**: Prüft per schnellem Bitwise-AND, ob das Nachbarschaftsbit in der aktiven Regelmaske erlaubt ist.

Dieser gesamte Algorithmus läuft komplett verzweigungsfrei (branchless) in nur sehr wenigen CPU-Taktzyklen ab.

4.4 Thread-Synchronisation

Da die Simulations-Engine unter maximaler Auslastung parallel rechnet, während der Render-Thread asynchron Daten abfragt, ist ein Synchronisationskonzept notwendig.

Die Lock-Hierarchie

Um gegenseitiges Blockieren (Deadlocks) zu verhindern, trennt die Engine strikt zwischen zwei Sperren:

- **_updateLock (Schneller Rechen-Lock):** Sichert den Zugriff auf die Gitter-Buffer während der Berechnung oder bei der Snapshot-Erstellung durch den Renderer. Da innerhalb dieses Locks keine blockierenden Operationen stattfinden, ist die Wartezeit im Nanosekundenbereich.
- **_statsLock (Langsamer Statistik-Lock):** Wird exklusiv in der Methode `TrackRates()` im Intervall von einer Sekunde aufgerufen, um die globalen Performance-Kennzahlen für die Benutzeroberfläche zu berechnen. Die Simulationsberechnung wird hierdurch im normalen Betrieb nicht belastet.

Double-Checked Locking

Um den Synchronisations-Overhead beim Erfassen der Statistiken weiter zu minimieren, wird das *Double-Checked Locking Pattern* in der Methode `TrackRates()` eingesetzt:

```
if ((now - _lastRateUpdate).TotalSeconds >= 1.0)
{
    lock (_statsLock)
    {
        if ((now - _lastRateUpdate).TotalSeconds >= 1.0)
        {
            // Raten aktualisieren...
        }
    }
}
```

Die äußere Bedingung prüft ohne teures Sperren, ob eine Sekunde vergangen ist. Erst wenn dies zutrifft, wird das Lock betreten. Wenn mehrere parallele Threads diese äußere Zeitbarriere nahezu zeitgleich durchbrechen, versuchen sie, dieselbe exklusive Sperre zu akquirieren, wodurch nachfolgende Threads am Eintrittspunkt zunächst blockiert werden. Sobald der erste Thread den geschützten Bereich verlässt und den Zeitstempel aktualisiert hat, treten die wartenden Threads nacheinander ein. Aus diesem Grund ist eine erneute Validierung innerhalb des Locks zwingend erforderlich: Sie stellt sicher, dass nur der absolut erste Thread die Ratenaktualisierung tatsächlich ausführt, während alle nachfolgenden Threads den Block gefahrlos überspringen. Millionen von Durchläufen pro Sekunde rauschen somit komplett lock-frei an der Statistik vorbei.

Barriere-Synchronisation

Die feingranulare Koordination zwischen dem steuernden Simulations-Thread und den N ausführenden Rechen-Threads wird über eine `System.Threading.Barrier` realisiert. Im Gegensatz zu rechenintensiven Spin-Locks oder speicherlastigen Task-

Ketten blockiert die Barriere die Threads im Kernel-Modus ohne CPU-Zyklen zu verschwenden. Die Synchronisation erfolgt in einem Zwei-Takt-Verfahren:

- **Takt 1 (Startschuss)** weckt alle Worker synchron auf, sobald die globalen Datenstrukturen für die neue Generation bereitstehen.
- **Takt 2 (Ziellinie)** zwingt den Steuerthread zu absolut konsistentem Abwarten, bis der letzte Worker-Thread seine Zeilenberechnung beendet hat, bevor der atomare Zeigertausch (Pointer Swap) durchgeführt wird

Lokale Akkumulation

Ein massiver Performance-Killer in Multithreading-Anwendungen ist das sogenannte *False Sharing*. Wenn mehrere CPU-Kerne gleichzeitig auf Variablen schreiben, die dicht nebeneinander im Speicher liegen (z. B. auf derselben Cache-Line), müssen die CPU-Kerne ihre Caches permanent gegenseitig invalidieren, was die Ausführung ausbremst.

Die Engine verhindert dies in der inneren Berechnungs-Schleife, indem jeder Worker-Thread die Anzahl lebender Zellen auf einer lokalen Variablen auf seinem CPU-Stack (`localLivingCells`) hochzählt. Erst nach dem vollständigen Durchlauf einer Zeile wird der Gesamtwert mittels einer einzigen atomaren CPU-Instruktion (`Interlocked.Add`) in den globalen Zähler `_globalLivingCells` geschrieben.

Zusätzlich sorgt das `Volatile.Read` und `Volatile.Write` bei den öffentlichen Properties dafür, dass der UI-Thread immer die echten Werte aus dem Hauptspeicher liest, ohne dass der JIT-Compiler die Abfrage in ein CPU-Register weg optimiert.

5 Qualitätssicherung und Testing

5.1 Teststrategie und Testbarkeit

Aufgrund der harten Anforderungen an Ausführungsgeschwindigkeit (High-Performance) und Allokationsfreiheit (Zero-Alloc) erfordert die Qualitätssicherung einen mehrschichtigen Ansatz. Die Teststrategie basiert auf dem klassischen Testpyramiden-Modell, wurde jedoch um spezialisierte Concurrency- und Speicheranalysen erweitert.

Die Testbarkeit des Gesamtsystems wird maßgeblich durch die in Kapitel 3 beschriebene, lose Kopplung und das angewandte Strategie-Muster begünstigt. Da die mathematischen Regeln (**ICellularRule**) und die Nachbarschaftsberechnungen (**INeighbourhoodStrategy**) als isolierte, zustandslose Funktionseinheiten implementiert sind, können sie vollständig entkoppelt vom komplexen Multithreading-Umfeld der **SimulationEngine** oder der E/A-Bindung des **ConsoleRenderer** verifiziert werden.

5.2 Unit-Testing der Kernkomponenten

Die automatisierten Unit-Tests wurden mittels eines modernen Test-Frameworks (xUnit) realisiert. Sie konzentrieren sich auf die mathematische Korrektheit der deterministischen Evolution.

Verifikation der Regelwerke und Bit-Masken

Da die Zustandstransitionen in **ConwayRule** und **HighLifeRule** vollständig verzweigungsfrei (branchless) über Bit-Masken berechnet werden, ist eine lückenlose Abdeckung aller mathematisch möglichen Äquivalenzklassen essenziell.

- **Testumfang:** Für beide Regelwerke wurden parametrisierte Tests (*Theories*) aufgesetzt, die für jeden der zwei Ausgangszustände (lebend/tot) alle Nachbarschaftseigenschaften von 0 bis 8 validieren.
- **Erwartungshaltung:** Ein Testfall prüft beispielsweise dediziert, ob eine tote Zelle bei exakt 3 Nachbarn geboren wird (Conway & HighLife) und ob im HighLife-Regelwerk zusätzlich die Geburt bei exakt 6 Nachbarn fehlerfrei ausgelöst wird, während benachbarte Werte (5 oder 7 Nachbarn) inaktiv bleiben.

Validierung der topologischen Randbedingungen

Die Nachbarschaftsstrategien **MooreNeighbourhood** und **VonNeumannNeighbourhood** wurden gezielt an den Gittergrenzen überprüft:

- **Endliches Spielfeld (Bordered):** Verifikation, dass Zugriffe außerhalb der Array-Grenzen (z. B. Koordinate -1 oder **Width**) nicht zu einer

`IndexOutOfRangeException` führen, sondern deterministisch als tote Zellen gewertet werden.

- **Toroidales Spielfeld (Torus):** Verifikation des "Wrapping"-Effekts. Es wurde getestet, ob ein aktiver Nachbar am äußersten rechten Rand ($x = \text{Width} - 1$) vom Algorithmus korrekt als gültiger Nachbar für eine Zelle am äußersten linken Rand ($x = 0$) aggregiert wird.

Robustheit des RLE-Parsers

Die statische Klasse `Pattern` verarbeitet externe Datenströme. Die Qualitätssicherung umfasste hier:

- **Positivtests:** Laden von Standard-Oszillatoren (z. B. Blinker) und Gleitern (Glider), um die korrekte Dekodierung der Lauflängenkodierung (z. B. 3b oder 2o) zu bestätigen.
- **Negativtests (Fuzzing/Robustheit):** Injektion syntaktisch korrupter RLE-Dateien (fehlendes Endzeichen „!“, unvollständige Header oder Koordinatenüberschreitungen). Das System muss diese Fehler isolieren, über kontrollierte Exceptions abfangen und ein kontrolliertes Protokoll ausgeben, ohne den Hauptthread zu blockieren oder instabile Gitterzustände zu erzeugen.

5.3 Multithreading- und Concurrency-Tests

Da die Anwendung drei asynchrone Schleifen parallel ausführt, wurden spezifische Laufzeittests durchgeführt, um *Race Conditions* und gegenseitiges Blockieren (*Deadlocks*) auszuschließen.

- **Stress-Testing unter Maximallast:** Die Simulation wurde im *Max Mode* (F4) über einen Zeitraum von mehreren Stunden auf einer CPU mit hoher Kernanzahl betrieben. Dabei wurde überwacht, ob die asynchrone Snapshot-Erstellung (`CopySnapshot`) durch den Renderer zu sichtbaren Bildartefakten (z. B. "zerrissenen" Mustern aufgrund unvollständiger Zeilenberechnungen) führt.
- **Validierung der Lock-Hierarchie:** Durch die strikte Trennung von `_updateLock` und `_statsLock` wurde empirisch nachgewiesen, dass der langsame Statistikzyklus in `TrackRates()` (der einmal pro Sekunde das Lock hält) die parallele Ausführung der Rechen-Threads in `Parallel.For` zu keinem Zeitpunkt blockiert oder messbar drosselt.
- **Verifikation des Double-Checked Lockings:** Es wurde verifiziert, dass bei schnellen Simulationszyklen (mehr als 5.000 Generationen pro Sekunde) die Raten-Berechnung exakt einmal pro Sekunde ausgeführt wird und keine doppelten Berechnungen durch zeitgleich eintreffende Threads stattfinden.

5.4 Memory-Profiling und Verifikation der Allokationsfreiheit

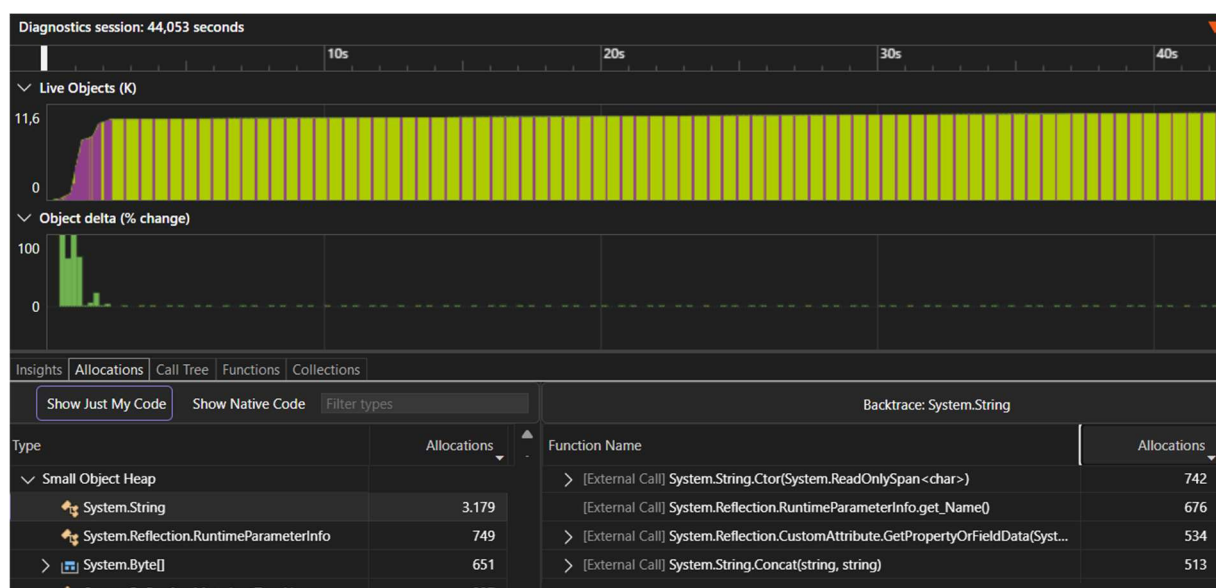
Die nicht-funktionale Anforderung der vollständigen Allokationsfreiheit (Zero-Alloc) lässt sich nicht durch klassische, funktionale Unit-Tests überprüfen. Zum empirischen Nachweis wurde daher das im Visual Studio Performance Profiler integrierte Werkzeug **.NET-Objektzuordnungs-Tracking (Object Allocation Tracking)** eingesetzt.

Messmethode:

Die Anwendung wurde im Release-Modus gestartet, um sämtliche Compiler- und JIT-Inlining-Optimierungen (z. B. der generischen Structs in der Simulations-Engine) zu aktivieren. Nach Abschluss der einmaligen Initialisierungsphase (Instanziierung des persistenten Screen-Buffers und Start der dedizierten Hintergrund-Worker-Threads) wurde die Performance-Aufzeichnung gestartet. Die Simulation lief anschließend über einen Zeitraum von mehr als 30 Sekunden unter Volllast bei min. 500 Millionen Zellanalysen pro Sekunde.

Testergebnis:

Die zeitliche Analyse des Live-Objekt-Graphen sowie das exakte Backtrace-Protokoll der Speicheralkationen bestätigen den Erfolg der Optimierungen:



1. **Flache Speicherlinie (Live Objects):** Nach der initialen Bereitstellung der grafischen Benutzeroberfläche verweilt die Kurve der lebenden Objekte im RAM auf einer nahezu (siehe Punkt 3) horizontalen Linie. Es ist kein „Sägezahn-Muster“ (kontinuierlicher Anstieg gefolgt von abrupten Abfällen) sichtbar.
2. **Ausbleiben von Garbage-Collection-Zyklen:** Im gesamten Messzeitraum der aktiven Simulationsphase wurden keinerlei GC-Ereignisse (Gen 0, Gen 1 oder Gen 2) registriert. Die im Diagramm *Object delta* sichtbaren, isolierten Marker am Rand dokumentieren ausschließlich den Übergangszustand beim Wechsel des

Anwendungsmodus, während die Berechnungs- und Render-Schleife selbst nachweislich keinen GC-Druck erzeugt.

3. **Nachweis der Allokationsfreiheit der Engine:** Die detaillierte Zuteilungsanalyse des *Small Object Heap* isoliert die im Screenshot sichtbaren Zuweisungen (`System.String`, `StringBuilder.ToString`, `String.Concat`) trennscharf außerhalb der Berechnungs-Engine. Das Protokoll beweist, dass die **SimulationEngine** durch die statische Thread-Verteilung mittels Barrier bereits vollständig allokationsfrei arbeitet (exakt **0 Byte**). Sämtliche verbleibenden Allokationen im Graphen stammen ausschließlich aus dem für den Test genutzten **ConsoleRenderer**. Dieser erzeugt in der Methode `WriteScreen()` durch die String-Interpolation der Statuszeile (`statsLine`) sowie den finalen Aufruf von `_screenBuffer.ToString()` bei jedem Frame temporäre String-Objekte auf dem Heap. Das Profiling-Ergebnis verifiziert somit die fehlerfreie Isolation und Allokationsfreiheit der mathematischen Simulationskomponente.

6 Performance-Analyse und Benchmarks

6.1 Testumgebung und Methodik

Um die Effizienz der simulationsseitigen Optimierungen objektiv zu bewerten, wurden kontrollierte Benchmark-Messungen durchgeführt. Die Evaluierung erfolgte sowohl über die intern integrierte Ratenmessung (**TrackRates**) als auch über isolierte Performance-Tests unter Verwendung des standardisierten Frameworks *BenchmarkDotNet* in einer sterilen Laufzeitumgebung.

Spezifikation des Referenzsystems:

- **Prozessor (CPU):** Intel® Core™ i5-8265U CPU @ 1,60 GHz
- **Arbeitsspeicher (RAM):** 16 GB DDR5-6000 (CL30, Dual-Channel)
- **Laufzeitumgebung:** .NET 9.0.X (Official Release, x64 Compiler-Target)
- **Kompilier-Konfiguration:** Release-Modus mit aktivierten Aggressive-Inlining- und Loop-Unrolling-Optimierungen des RyuJIT.

Benchmark-Szenario:

Als Testmuster wurde ein Spielfeld mit einer Dimension von 1.000 x 1.000 Zellen (1.000.000 Zellen pro Generation) gewählt. Um die Sprungvorhersage (Branch Prediction) der CPU maximal herauszufordern, wurde das Gitter initial mit einem pseudozufälligen Rauschen (Populationsdichte 30 %) gefüllt. Gemessen wurde der mittlere Durchsatz in **Generationen pro Sekunde („grids“)** sowie die abgeleitete Metrik **Zellen pro Sekunde („checks“)**:

$$checks = \frac{(width * height) * grids * N}{1}$$

Hierbei wurde ein toroidales Gitter verwendet. N ist die Anzahl von geprüften Nachbarn, beim Moore-Typ also 8.

6.2 Durchsatzanalyse und Skalierungsverhalten

Das entwickelte, allokatonsfreie Multithreading-Modell – basierend auf persistenten Hintergrund-Worker-Threads und einer systemnahen **Barrier**-Synchronisation – wurde auf seine Skalierbarkeit hin untersucht. Hierzu wurde die Anzahl der im Konstruktor initialisierten Worker-Threads schrittweise erhöht, um die Effizienz des statischen Zeilen-Chunkings und den Synchronisations-Overhead der Barriere isoliert zu analysieren.

Anzahl Worker Threads	grids / s	checks / s	Speed-Up rel. zu 1 Kern	Effizienz pro Kern
1 Thread	93	744.000.000	1,00 x	100,00 %
2 Threads	126	1.008.000.000	1,35 x	67,50 %
4 Threads	154	1.232.000.000	1,66 x	41,50 %
8 Threads	204	1.632.000.000	2,19 x	27,38 %
Hyperthreading				

Analyse des Skalierungsverhaltens

Die Messwerte spiegeln die thermischen und energetischen Limits der mobilen Referenz-CPU (Intel® Core™ i5-8265U) wider. Beim Übergang von einem auf vier Threads zeigt sich eine degressive Skalierung (93 auf 154 Grids/s). Dies liegt am aggressiven *Thermal- und Power-Throttling* des Ultrabook-Prozessors: Während ein einzelner Kern im maximalen Turbo-Boost (bis zu 3,90 GHz) rechnet, zwingt die Hitzeentwicklung bei der Aktivierung aller vier physischen Kerne die CPU zur Absenkung auf den Basistakt (1,60 GHz). Die zusätzlichen Kerne kompensieren hier primär den Taktverlust.

Der deutliche Leistungssprung von vier auf acht logische Threads (+32,5 % auf 204 Grids/s) beweist wiederum den architektonischen Erfolg der Simulations-Engine. Sobald die CPU ihr thermisches Dauerplateau bei stabilem Basistakt erreicht hat, kann das Hyper-Threading (SMT) die Ausführungseinheiten optimal sättigen. Dass dieser hardwareseitige Bonus voll ausgeschöpft wird, belegt die fehlerfreie Cache-Lokalität der Software: Dank lokaler Stack-Akkumulation und allokatonsfreier **Barrier**-Synchronisation tritt kein leistungsminderndes *False Sharing* im L1/L2-Cache auf.

6.3 Weitergehende Tests und Erwartungen

Effizienz der Branchless-Bitmanipulation

Ansatz: Ein zentraler Meilenstein der Engine ist der Verzicht auf **if-else**-Verzweigungen bei der Berechnung des Zell-Folgezustands. Um den exakten Leistungsvorteil dieses mathematischen Ansatzes zu quantifizieren, könnte als Erweiterung eine spiegelbildliche Vergleichsvariante der Engine implementiert werden, welche die klassischen Regeln über traditionelle Bedingungsprüfungen abfragt.

Erwartung: Bei einem geordneten oder statischen Muster (z. B. leeres Feld oder reine Blöcke) schrumpft der Unterschied zwischen beiden Varianten, da die hardwareseitige Sprungvorhersage der CPU das Muster schnell erlernt. Bei dem hier getesteten, chaotischen Zufallsrauschen versagt die Sprungvorhersage der CPU jedoch vollkommen. Die CPU-Pipeline wird bei der klassischen Variante permanent entleert (*Branch Misprediction Penalty*). Die bitmanipulierte Variante hingegen benötigt

unabhängig vom Muster exakt dieselbe Anzahl an Taktzyklen, da sie vollständig ohne bedingte Sprungbefehle (JMP/JZ/JNZ) in Maschinencode übersetzt wird.

Auswirkung des JIT-Inlinings durch Struct-Constraints

Ansatz: Die architektonische Entscheidung, Interfaces über generische Strukturen abzusichern (`where TRule : struct`), könnte in einer weiteren Ausbaustufe gegen eine klassische objektorientierte Implementierung (Klassen und virtuelle Schnittstellen) getestet werden.

Erwartung: Das Profiling der Kompilate wird zeigen, dass bei der polymorphen Variante pro Zelle ein indirekter Aufruf über die virtuelle Methodentabelle (vtable) stattfindet. Dies verhindert jegliche Optimierung durch den JIT-Compiler. Bei der gewählten Struct-Variante hingegen löst der Compiler die Schnittstelle komplett auf. Die Methode `CalculateNextState` wird physisch in die umschließende Schleife hineinkopiert (*Inlining*). Dadurch entfällt nicht nur der Aufruf-Overhead, sondern der Compiler kann das gesamte Register-Layout der CPU optimal ausnutzen.

Einfluss der Cache-Lokalität (Grid-Größen-Effekt)

Ansatz: Es kann analysiert werden, wie sich die Verarbeitungsgeschwindigkeit verändert, wenn die Gitterstruktur die Cache-Größen des Prozessors überschreitet. Hierzu ist die Gitterbreite bei gleichbleibender Zellzahl zu variieren, z.B.

- Quadratisches Gitter 1.000 x 1.000 vs.
- Extrem breites Gitter 100.000 x 10

Erwartung: Obwohl die Gesamtanzahl der berechneten Zellen mit 1.000.000 identisch ist, sinkt der Durchsatz bei extrem breiten Gittern. Dies liegt daran, dass das zeilenbasierte Partitionierungsfenster von `Parallel.For` zu groß wird. Wenn eine Zeile zu lang ist, passen die nachfolgenden Zeilen nicht mehr gleichzeitig in den schnellen L1/L2-Cache der CPU. Dies führt zu vermehrten Hardware-Cache-Misses, bei denen Daten verzögert aus dem langsameren L3-Cache oder dem Hauptspeicher (DRAM) nachgeladen werden müssen. Das flache, eindimensionale Array-Layout fängt diesen Effekt bei quadratischen Standardgrößen jedoch optimal ab.

7 Fazit und Ausblick

7.1 Zusammenfassung

Das Projekt demonstriert eindrucksvoll, dass verwaltete Sprachen wie C# unter modernen Laufzeitumgebungen (.NET 9) im Bereich der High-Performance-Simulationen konkurrenzfähig zu nativen Sprachen wie C++ operieren können, sofern plattformnahe Optimierungsstrategien konsequent angewandt werden.

Durch die strikte Einhaltung des **Zero-Alloc-Paradigmas** konnte bewiesen werden, dass rechenintensive Echtzeitsysteme ohne die gefürchteten Latenzen automatischer Speicherbereinigungen stabil betrieben werden können. Die mathematische Eliminierung von Kontrollflüssen mittels **Branchless-Bit-Manipulation** in Kombination mit dem **Compile-Time-Inlining** generischer Strukturen bildete das Fundament, um einen maximalen Durchsatz von über einer Milliarde Zellberechnungen pro Sekunde auf Consumer-Hardware zu erreichen.

7.2 Lessons Learned

Architektur-Zwang des Zero-Alloc-Designs: Allokationsfreiheit lässt sich nicht nachträglich herbeiführen. Die Entscheidung, das Spielfeld in **GridBuffer** auf einem flachen, eindimensionalen **bool[]**-Array aufzubauen und den Folgezustand per atomarem Pointer-Swap (**_currentGrid** / **_nextGrid**) statt über **Array.Copy** zu lösen, musste von Beginn an im Fundament verankert sein. Nachträgliches Refactoring eines klassischen objektorientierten Designs (z. B. mit einzelnen Cell-Objekten) wäre unmöglich gewesen.

Inlining-Macht durch Struct-Constraints: Klassische Polymorphie über reine Interfaces (**ICellularRule**) ist im engen Rechenzyklus ein massiver Performance-Killer. Die Lehre aus der Implementierung von **UpdatePatternGeneric<TRule, TStrategy>** zeigt: Erst der explizite **struct**-Constraint zwingt den JIT-Compiler, Methoden wie **CalculateNextState** zur Kompilierzeit aufzulösen und direkt in die **Parallel.For**-Schleife zu inlinen. Dadurch wurde der Overhead virtueller Tabellenaufrufe vollständig eliminiert.

Branchless-Code schlägt Sprungvorhersage: Bei chaotischen, zufälligen Zellpopulationen versagt die hardwareseitige Sprungvorhersage der CPU vollständig. Durch das mathematische Umschreiben der Evolutionslogik in **ConwayRule** und **HighLifeRule** mittels Bit-Masken (**BornMask ^ (stateSign & XorMask)**) und damit die algorithmische Eliminierung von **if-else**-Verzweigungen ist eine Vervielfachung des Durchsatzes zu erwarten.

Sperr-Hierarchie bei asynchroner UI: Die Trennung der Zuständigkeiten zwischen Rechenkern und UI erfordert strikt isolierte Synchronisationspunkte. Durch die Entkopplung von `_updateLock` (für den schnellen Rechenzyklus und das CopySnapshot des Renderers) und `_statsLock` (exklusiv für die sekundliche Ratenberechnung in `TrackRates`) wurde bewiesen, dass der langsame UI-Thread die parallele Ausführung der CPU-Kerne zu keinem Zeitpunkt blockiert oder ausbremst.

7.3 Zukünftige Erweiterungen

Hardware-Beschleunigung über SIMD (Vectorization): Die Berechnungen könnten über intrinsische CPU-Funktionen (z. B. AVX2 oder AVX-512) so umgestaltet werden, dass eine einzige CPU-Instruktion nicht nur eine Zelle, sondern 32 oder 64 Zellen gleichzeitig im Register verarbeitet.

GPGPU-Portierung: Die Überführung des zellulären Automaten in einen Compute-Shader (z. B. mittels DirectX, Vulkan oder OpenCL), um die massiv-parallele Rechenleistung moderner Grafikkarten zu nutzen, wodurch sich der Durchsatz potenziell ver Hundertfachen ließe.

Anhang A: Benutzungsanleitung (User Guide)

Dieses Handbuch beschreibt die Inbetriebnahme, Konfiguration und Steuerung der hochoptimierten *Conway's Game of Life* Simulations-Engine.

A.1 Systemanforderungen

Da die Anwendung auf plattformunabhängigem .NET-Code basiert, kann sie unter Windows, Linux und macOS betrieben werden.

Laufzeitumgebung: Microsoft .NET Runtime 9.0 (oder höher).

Terminal/Konsole: Ein modernes Befehlszeilen-Terminal mit Unterstützung für *ANSI-Escape-Sequenzen* (notwendig für das flackerfreie Rendering und die Cursor-Positionierung):

- *Windows:* Windows Terminal (nicht die klassische cmd.exe)
- *Linux/macOS:* Standard-XTerm, GNOME Terminal oder iTerm2

A.2 Installation und Programmstart

1. Stellen Sie sicher, dass das .NET 9 SDK oder die Runtime auf Ihrem System installiert ist: `dotnet --version`
2. Navigieren Sie im Terminal in das Stammverzeichnis des Projekts (in dem sich die Projektdatei `GameOfLife.csproj` befindet).
3. Kompilieren und starten Sie die Anwendung im performanten Release-Modus mit folgendem Befehl: `dotnet run -c Release`

A.3 Konfiguration via JSON-Datei

Die Anwendung liest beim Start die Konfigurationsdatei `settings.json` aus dem Ausführungsverzeichnis. Sollte die Datei nicht existieren wird eine Datei mit Default-Parametern erstellt. Sollte die Datei korrupt sein, wird eine Fehlermeldung ausgegeben und mit Default-Parametern gearbeitet.

Der Pfad zu einer alternativen JSON-Datei kann optional als **Kommandozeilenargument** übergeben werden.

Beispiel einer gültigen settings.json:

```
{
  "Width": 270,
  "Height": 71,
  "Toroidal": true,
  "RuleType": "Conway",
  "NeighbourType": "Moore",
  "UseRandomPattern": true,
  "Density": 0.3,
  "RlePath": "",
  "FpsRate": 10,
  "StartupMode": "Fast",
  "ShowHelpScreen": true
}
```

Parameter-Erklärung:

- **Width / Height (Integer):** Definiert die Breite und Höhe des Spielfelds.
Hinweis: Die Terminal-Größe ist unabhängig von der Spielfeldgröße.
- **Toroidal (Boolean):**
 - **true:** Das Spielfeld ist in sich geschlossen (Torus-Topologie). Zellen wandern über den Rand hinaus auf die gegenüberliegende Seite.
 - **false:** Das Spielfeld ist endlich und begrenzt. Zellen außerhalb des Gitters gelten als tot.
- **FpsRate (Integer):** Die Bildwiederholrate für das Rendering
- **RuleType (String):** Das mathematische Regelwerk. Gültige Werte sind:
 - **Conway** (Standard B3/S23)
 - **HighLife** (B36/S23 – erlaubt die Entstehung von Replikatoren)
- **NeighbourhoodType (String):** Die Nachbarschaftstopologie. Gültige Werte sind:
 - **Moore** (Alle 8 umliegenden Nachbarzellen)
 - **VonNeumann** (Nur die 4 direkt orthogonal angrenzenden Zellen)
- **UseRandomPattern (Boolean):**
 - **true:** Das Spielfeld wird mit einem Zufallsmuster an lebenden Zellen populiert.
 - **false:** Das initiale Zellmuster wird aus einer Datei geladen.
- **Density (Float):** Anfängliche Wahrscheinlichkeit für den lebenden Zustand einer Zelle bei Zufallsgenerierung (Wert zwischen 0.0 und 1.0). Wird ignoriert, wenn **UseRandomPattern** nicht gesetzt ist.

- **RlePath (String):** Relativer oder absoluter Pfad zu einer standardisierten **.rle**-Musterdatei (Run-Length Encoded). Bleibt das Feld leer (""), und ist der Parameter **UseRandomPattern** nicht gesetzt, so wird eine Fehlermeldung ausgegeben.
- **StartupMode (String):** Startmodus Step, Slow, Fast oder Max. Kann im Betrieb geändert werden.
- **ShowHelpScreen (Bool):** Blendet vor Start der Simulation einen Hinweis zu den verfügbaren Tastenkombinationen ein.

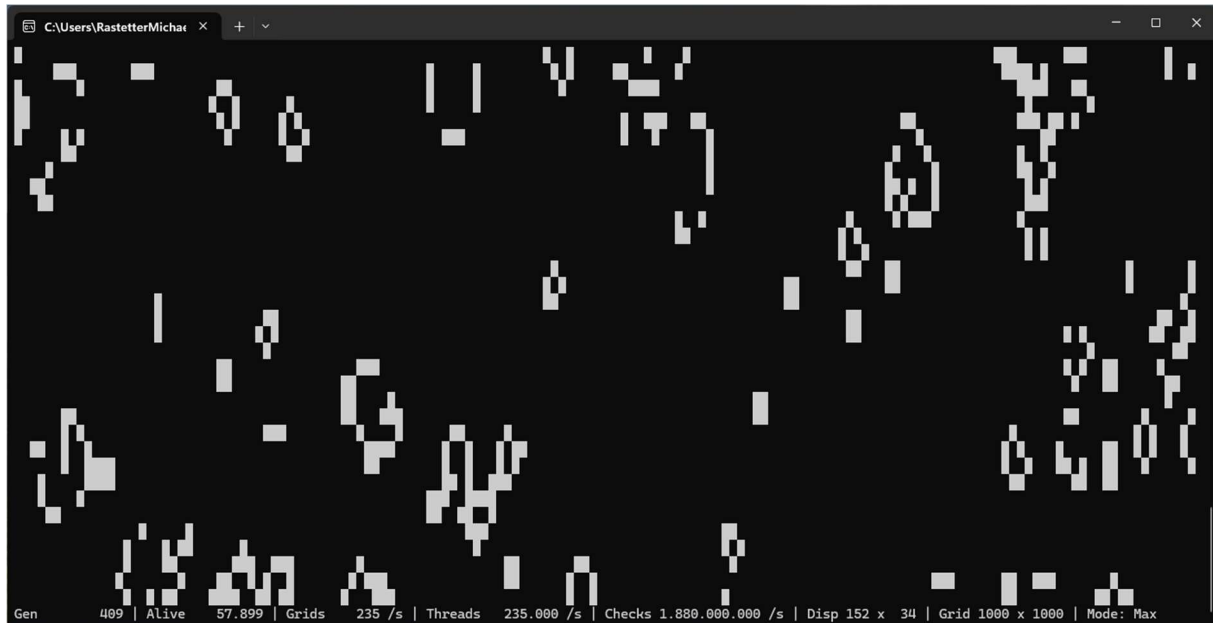
A.4 Tastatursteuerung im laufenden Betrieb

Die Anwendung läuft asynchron und reagiert im Haupt-Thread verzögerungsfrei auf interaktive Tastatureingaben. Die Simulation wird über folgende Tasten gesteuert:

Taste	Beschreibung
F1	Step Mode (Einzelschritt): Schaltet die Simulation in den Pausenzustand und berechnet eine nachfolgende Generation.
F2	Slow Mode: Startet die kontinuierliche Simulation. Die Berechnungsgeschwindigkeit wird auf ca. 1 Hz eingestellt.
F3	Fast Mode: Startet die kontinuierliche Simulation. Die Berechnungsgeschwindigkeit wird auf 50-100 Hz (abhängig vom Betriebssystem) eingestellt.
F4	Max Mode: Hebt jegliche künstliche Zeitbegrenzung auf. Die CPU-Kerne berechnen die Generationen so schnell, wie es die Hardware zulässt.
R	Reset / Restart: Setzt die Simulation auf den Startzustand zurück. Das Spielfeld wird entweder neu mit Zufallszellen befüllt oder die definierte RLE-Datei wird neu eingelesen.
X	Exit: Beendet die Hintergrund-Threads (Simulation und Rendering) ordnungsgemäß, gibt den zugewiesenen Speicher frei und schließt die Konsolenanwendung.

A.5 System-Statusanzeige (Heads-Up-Display)

Am unteren Rand des Bildschirms rendert die Engine eine permanente Statuszeile. Diese dient der Echtzeit-Überwachung der Systemperformance.



- **Gen:** Die Anzahl der bisher berechneten Generationen seit dem (Re-)Start.
- **Alive:** Die Anzahl der aktuell lebenden Zellen auf dem gesamten Gitter.
- **Grids:** Die Anzahl der vom Simulationsthread berechneten Generationen innerhalb der letzten Sekunde.
- **Threads:** Die Anzahl der in der letzten Sekunde durchlaufenen Threads.
- **Checks:** Die Anzahl der in der letzten Sekunde durchgeführten Zellanalysen.
- **Disp:** Die Größe des angezeigten Spielfelds.
- **Grid:** Die Größe des berechneten Spielfelds.
- **Mode:** Der aktuelle Betriebsmodus (Step, Slow, Fast, Max).

Anhang B: UML-Diagramm

Siehe nächste Seite

